In [ ]:
```python
import marimo as mo
from pathlib import Path
import contextlib
import io
import time
import inspect
import textwrap
```

Exercício 3 — Verificação formal de uma multiplicação

Este documento explica o trabalho implementado em **exe3.py**, apresenta o modelo formal e analisa as obrigações verificadas pelo Z3. O programa não se limita a calcular produtos: procura contraexemplos às propriedades do algoritmo.

O trabalho foi dividido em quatro partes: **A)** construção do sistema de transição, **B)** invariante, **C)** segurança e **D)** terminação. Os testes principais usam palavras de 8 e 16 bits, interpretadas como valores **sem sinal**.

Porque escolhemos a biblioteca Z3

Uma das razões da escolha foi o Z3 ter sido referenciado para realizar o trabalho. Seguimos essa indicação e usámos a interface Python, Z3Py, para escrever o modelo e executar as verificações no mesmo ambiente do código.

O Z3 é um solver SMT: verifica se um conjunto de fórmulas lógicas tem solução, combinando lógica com teorias como inteiros e vetores de bits. Estas capacidades ajustam-se ao exercício pelas seguintes razões:

- **Representação da aritmética de máquina:** `BitVec` permite modelar palavras de `n` bits e o comportamento modular, incluindo o overflow. `Extract`, `LShR`, `ULT` e `ZeroExt` representam as operações específicas usadas na implementação.
- **Modelação do controlo:** `Int`, `And`, `Or` e `Implies` permitem descrever o contador de programa, as guardas, as transições e os invariantes.
- **Procura simbólica de contraexemplos:** escrevemos as hipóteses e a negação da propriedade. `Solver.check()` determina se essa combinação é possível. Assim, verificamos as obrigações para os valores abrangidos pelas fórmulas.
- **Análise das respostas:** `sat` permite obter um contraexemplo com `model()`; `unsat` conclui a obrigação; `unknown` exige tratar a prova como inconclusiva. O timeout permite limitar o tempo de cada consulta.

Por exemplo, `Init ∧ ¬R` pergunta se existe um estado inicial que viole o invariante. Uma resposta `unsat` prova a base da indução. O mesmo princípio aplica-se à preservação pelas transições e à descida do ranking de terminação. As fórmulas e as hipóteses são definidas por nós; o Z3 verifica as consultas.

As operações e a utilização da interface estão documentadas no guia oficial de Z3Py e no guia de bit-vectors.

A seguir, cada explicação é acompanhada pelo bloco de código correspondente, pela ordem da implementação. Todo o código está incluído neste ficheiro Marimo e é apresentado também em modo apresentação.

In [ ]:
```python
def verificar(n, timeout_ms=10000):
    from z3 import (And, BV2Int, BitVec, BitVecVal, BitVecs, Extract, If,
Implies, Int, LShR, Not, Or, Solver, ULT, ZeroExt, sat, simplify, unknown,
unsat)

    # A implementação considera larguras positivas e pares.
    if n <= 0 or n % 2 != 0:
        raise ValueError("n deve ser positivo e par")

    resultados = []

    def provar(nome, *formulas):
        # Cada consulta procura um contraexemplo à obrigação indicada.
        s = Solver()
        s.set(timeout=timeout_ms)
        s.add(*(simplify(f, som=True) for f in formulas))
        resultado = s.check()
        resultados.append(resultado == unsat)
        print(f"{nome}: {resultado}")
        if resultado == sat:
            print("  Contraexemplo:", s.model())
        elif resultado == unknown:
            print("  Prova inconclusiva:", s.reason_unknown())
        return resultado == unsat

    print(f"\n========== TESTES PARA n = {n} ==========")

    # ALINEA A - CONSTRUCAO DO SFOTS

    # Estado atual
    a = BitVec(f'a_{n}', n)
    b = BitVec(f'b_{n}', n)
    x = BitVec(f'x_{n}', n)
    y = BitVec(f'y_{n}', n)
    z = BitVec(f'z_{n}', n)
    pc = Int(f'pc_{n}')

    # Estado seguinte
    a1 = BitVec(f'a1_{n}', n)
    b1 = BitVec(f'b1_{n}', n)
    x1 = BitVec(f'x1_{n}', n)
```

```
    y1 = BitVec(f'y1_{n}', n)
    z1 = BitVec(f'z1_{n}', n)
    pc1 = Int(f'pc1_{n}')

    # pc: 1 entrada; 2 teste; 3 deslocamento; 4 soma;
    #     5 erro de deslocamento; 6 final; 7 erro de soma.
    # Mantém-se o caso zero: o modelo cobre também entradas não negativas.
    Init = And(pc == 1, x == a, y == b, z == BitVecVal(0, n))

    T1 = And(pc == 1, pc1 == 2, a1 == a, b1 == b, x1 == x, y1 == y, z1 ==
z)

    T2 = And(pc == 2, y == 0, pc1 == 6, a1 == a, b1 == b, x1 == x, y1 == y,
z1 == z)

    T3 = And(pc == 2, y != 0, Extract(0, 0, y) == 0, pc1 == 3, a1 == a, b1
== b, x1 == x, y1 == y, z1 == z)

    T4 = And(pc == 2, y != 0, Extract(0, 0, y) == 1, pc1 == 4, a1 == a, b1
== b, x1 == x, y1 == y, z1 == z)

    OverflowLeft = Extract(n-1, n-1, x) == 1

    T5 = And(pc == 3, OverflowLeft, pc1 == 5, a1 == a, b1 == b, x1 == x, y1
== y, z1 == z)

    T6 = And(pc == 3, Not(OverflowLeft), pc1 == 2, a1 == a, b1 == b, x1 ==
x << 1, y1 == LShR(y, 1), z1 == z)

    OverflowAdd = ULT(x + z, z)

    T7 = And(pc == 4, OverflowAdd, pc1 == 7, a1 == a, b1 == b, x1 == x, y1
== y, z1 == z)

    T8 = And(pc == 4, Not(OverflowAdd), pc1 == 2, a1 == a, b1 == b, x1 ==
x, y1 == y - 1, z1 == z + x)

    T9 = And(pc == 6, pc1 == 6, a1 == a, b1 == b, x1 == x, y1 == y, z1 ==
z)

    T10 = And(Or(pc == 5, pc == 7), pc1 == pc, a1 == a, b1 == b, x1 == x,
y1 == y, z1 == z)

    Trans = Or(T1, T2, T3, T4, T5, T6, T7, T8, T9, T10)

    Error = Or(pc == 5, pc == 7)


    # ALINEA B - INVARIANTE PRINCIPAL (k-indução com k = 1)
    # P exprime igualdade módulo 2**n. Q reforça as hipóteses de controlo.

    P = (x*y + z == a*b)

    Q = And(Implies(pc == 3, And(y != 0, Extract(0, 0, y) == 0)),
Implies(pc == 4,Extract(0, 0, y) == 1))
```

```
    R = And(P, Q)

    P1 = (x1*y1 + z1 == a1*b1)

    Q1 = And(Implies(pc1 == 3, And(y1 != 0, Extract(0, 0, y1) == 0)),
Implies(pc1 == 4, Extract(0, 0, y1) == 1))

    R1 = And(P1, Q1)

    provar("Base reforçada (k=1)", Init, Not(R))

    transicoes = [T1, T2, T3, T4, T5, T6, T7, T8, T9, T10]

    for i, t in enumerate(transicoes, 1):
        if i not in (6, 8):
            provar(f"T{i}", R, t, Not(R1))

    # T6 e T8 são verificadas por composição: relações entre os estados
    # e identidades algébricas. Isto evita consultas com produtos difíceis
    # de resolver, sem substituir verificações por mensagens de sucesso.
    estrutura_left = And(
        x1 == 2*x, y == 2*y1, z1 == z,
        a1 == a, b1 == b, Q1
    )
    estrutura_right = And(
        x1 == x, y1 == y - 1, z1 == z + x,
        a1 == a, b1 == b, Q1
    )
    provar("T6: estrutura modular", Q, T6, Not(estrutura_left))
    provar("T8: estrutura modular", Q, T8, Not(estrutura_right))

    def provar_identidades(bits, contexto):
        u, v, w = BitVecs(f"u_{bits} v_{bits} w_{bits}", bits)
        # LEFT: y = 2*v e x' = 2*x, logo x'*y' + z' = x*y + z.
        provar(f"{contexto} T6: identidade", (2*u)*v + w != u*(2*v) + w)
        # RIGHT: x*(y-1) + (z+x) = x*y + z.
        provar(f"{contexto} T8: identidade", u*(v - 1) + (w + u) != u*v +
w)

    # As estruturas e estas identidades, juntamente com R, implicam R1.
    provar_identidades(n, "Invariante")

    # ALINEA C - PROVA DE SEGURANCA (k-indução com k = 1)

    limite = BitVecVal(2**(n//2), n)
    InitSafe = And(Init, ULT(a, limite), ULT(b, limite))

    def wide(v):
        return ZeroExt(n, v)

    # 2*n bits bastam para o produto de dois valores de n bits mais z:
    # (2**n-1)**2 + (2**n-1) < 2**(2*n). Assim, E é igualdade exata.
    E = (wide(x)*wide(y) + wide(z) == wide(a)*wide(b))
    E1 = (wide(x1)*wide(y1) + wide(z1) == wide(a1)*wide(b1))
    Error1 = Or(pc1 == 5, pc1 == 7)
```

```
SafeInv = And(E, Q, ULT(a, limite), ULT(b, limite), Not(Error))
SafeInv1 = And(E1, Q1, ULT(a1, limite), ULT(b1, limite), Not(Error1))

provar("Base safety (k=1)", InitSafe, Not(SafeInv))

for i, t in enumerate(transicoes, 1):
    if i not in (6, 8):
        provar(f"Safety T{i}", SafeInv, t, Not(SafeInv1))

# Nas transições sem overflow, as atualizações também são exatas
# quando estendidas a 2*n bits. Os restantes membros de SafeInv
# são preservados por a1=a, b1=b, Q1 e Not(Error1).
estrutura_left_exata = And(
    wide(x1) == 2*wide(x), wide(y) == 2*wide(y1),
    wide(z1) == wide(z), a1 == a, b1 == b, Q1, Not(Error1)
)
estrutura_right_exata = And(
    wide(x1) == wide(x), wide(y1) == wide(y) - 1,
    wide(z1) == wide(z) + wide(x),
    a1 == a, b1 == b, Q1, Not(Error1)
)
provar("Safety T6: estrutura exata", Q, T6, Not(estrutura_left_exata))
provar("Safety T8: estrutura exata", Q, T8, Not(estrutura_right_exata))
# As mesmas identidades a 2*n bits provam E => E1 em T6 e T8.
provar_identidades(2*n, "Safety")
provar("Invariante implica segurança", SafeInv, Error)

# ALINEA D - PROVA DE TERMINACAO
# Propriedade de animação: F(pc=5 ou pc=6 ou pc=7).
# Com InitSafe, a segurança exclui os erros e obtemos F(pc=6).
Terminal = Or(Error, pc == 6)
Dominio = And(pc >= 1, pc <= 7)
Dominio1 = And(pc1 >= 1, pc1 <= 7)

provar("Terminação: base do controlo", Init, Not(And(Dominio, Q)))
provar("Terminação: preservação do controlo",
       Dominio, Q, Trans, Not(And(Dominio1, Q1)))

def ranking(controle, multiplicador):
    terminal = Or(controle == 5, controle == 6, controle == 7)
    fase = If(controle == 1, 3, If(controle == 2, 2, 1))
    # BV2Int interpreta y sem sinal; a conta do ranking usa Int,
    # para que o próprio ranking não possa sofrer overflow.
    return If(terminal, 0, 3*BV2Int(multiplicador) + fase)

rho = ranking(pc, y)
rho1 = ranking(pc1, y1)
provar("Terminação: ranking não negativo", Dominio, rho < 0)
provar("Terminação: ranking diminui",
       Dominio, Q, Trans, Not(Terminal), Not(rho1 < rho))
# As guardas das transições cobrem todos os estados do domínio.
# Logo não há bloqueios; a descida nos naturais obriga a atingir
# um terminal. Os ciclos T9 e T10 apenas representam a paragem.

concluido = all(resultados)
if concluido:
```

```
        print(f"Todas as {len(resultados)} obrigações foram provadas para n={n}.")
    else:
        print(f"Verificação incompleta para n={n}: existem falhas ou resultados inconclusivos.")
    return concluido
```

In [ ]:
```
pasta_exe3 = Path(__file__).resolve().parent
```

In [ ]:
```
codigo_exe3 = textwrap.dedent(inspect.getsource(verificar))
_marcadores = ["    # ALINEA A", "    # ALINEA B", "    # ALINEA C", "    # ALINEA D", "    concluido = all(resultados)"]
_posicoes = [0] + [codigo_exe3.index(_m) for _m in _marcadores] + [len(codigo_exe3)]
blocos_codigo = [textwrap.dedent(codigo_exe3[_a:_b]).strip() for _a, _b in zip(_posicoes, _posicoes[1:])]
```

In [ ]:
```
mo.vstack([mo.md('## O que faz o algoritmo\n\nPartimos de `x=a`, `y=b` e `z=0`. Enquanto `y` não for zero:\n\n1. Se `y` for par, duplicamos `x` e dividimos `y` por dois.\n2. Se `y` for ímpar, somamos `x` a `z` e decrementamos `y`.\n3. Antes de atualizar, verificamos se o deslocamento ou a soma provocariam overflow.\n\nQuando `y=0`, o resultado está em `z`. Por exemplo, com `a=3` e `b=5`,\nos triplos `(x,y,z)` evoluem assim:\n`(3,5,0) → (3,4,3) → (6,2,3) → (12,1,3) → (12,0,15)`.\nAs transições de controlo entre estas atualizações também fazem parte do modelo.\n\n\n\n### Preparação e função de prova\n\nA função `verificar` recebe a largura e o timeout. Valida que a largura é positiva e par. A função interna `provar` cria um solver por obrigação, adiciona as fórmulas e regista se o resultado é `unsat`. Quando encontra um contraexemplo ou uma resposta inconclusiva, mostra os detalhes.'), mo.md("```python\n" + blocos_codigo[0] + "\n```")])
```

In [ ]:
```
mo.image(
    src=pasta_exe3 / "exe3_fluxograma.svg",
    alt="Fluxograma da multiplicação: inicialização, teste de y, ramos par e ímpar, paragem e erros de overflow.",
    width=720,
)
```

In [ ]:
```
mo.vstack([mo.md('## A — Construção do SFOTS\n\nO SFOTS é um sistema de transição simbólico de primeira ordem. O estado contém\n`a, b, x, y, z` (bit-vectors de `n` bits) e `pc` (inteiro que indica a fase).\nAs variáveis com sufixo `1` representam o estado seguinte. `a` e `b` permanecem\nconstantes em todas as transições.\n\n| pc | Significado | Transições |\n|---|---|---|\n| 1 | Entrada: `x=a`, `y=b`, `z=0` | T1 vai para o teste |\n| 2 | Teste de `y` | T2 termina se zero; T3 escolhe par; T4 escolhe ímpar |\n| 3 | Deslocamento | T5 deteta overflow; T6 atualiza `x` e `y` |\n| 4 | Soma | T7 deteta overflow; T8 atualiza `y` e `z` |\n| 5 | Erro de deslocamento | T10 mantém o estado |\n| 6 | Final normal | T9 mantém o estado |\n| 7 | Erro de soma | T10 mantém o estado |\n\n`Init` descreve os estados iniciais e `Trans` é a disjunção de T1 a T10.\nOs ciclos nos estados terminais representam a paragem do algoritmo.\n\n`Extract(n-1,n-1,x)==1` deteta a perda de um bit ao duplicar `x`.\n`ULT(x+z,z)` deteta o retorno modular da soma. `LShR(y,1)` faz um\ndeslocamento lógico,')])
```

```
introduzindo zero no bit superior.\n\n'), mo.md("```python\n" +
blocos_codigo[1] + "\n```")])
```

In [ ]:
```
mo.vstack([mo.md("## B — Invariante e indução com k=1\n\nA relação
principal é:\n\n$$P:\\quad xy+z=ab\\pmod {2^n}.$$\n\nAs operações sobre
bit-vectors são modulares. Por isso, esta igualdade, isoladamente,\nnão
demonstra que o resultado matemático cabe em `n` bits.\n\nReforçámos `P`
com `Q`: em `pc=3`, `y` é par e diferente de zero; em `pc=4`,\n`y` é ímpar.
O invariante usado é `R=P∧Q`. Este reforço fornece as
hipóteses\nnecessárias para provar as transições de atualização, incluindo
estados que a\nconsulta indutiva considera sem reconstruir o percurso desde
a entrada.\n\nA base procura um estado que satisfaça `Init ∧ ¬R`. O passo
procura um estado que\nsatisfaça `R ∧ Ti ∧ ¬R1`. Se a resposta for `unsat`,
não existe esse contraexemplo.\n\nT6 e T8 são tratadas por composição:
provamos as relações entre os estados e,\nseparadamente, as identidades
algébricas\n\n$$(2x)y'+z=x(2y')+z,\\qquad x(y-1)+(z+x)=xy+z.$$\n\nIsto
evita consultas diretas difíceis com produtos simbólicos. As identidades
e\nas relações estruturais implicam a preservação de `P`; não são execuções
concretas\ndo algoritmo nem uma única consulta monolítica ao passo
indutivo.\n\n"), mo.md("```python\n" + blocos_codigo[2] + "\n```")])
```

In [ ]:
```
mo.vstack([mo.md('## C — Segurança: excluir overflow\n\nA prova de
segurança restringe as entradas a\n\n$$0\\le a,b<2^{n/2}.$$\n\nPara 8 bits,
as entradas vão de 0 a 15; para 16 bits, de 0 a 255.\nO modelo inclui zero.
O produto máximo é, respetivamente, 225 e 65025,\nabaixo de
$2^n$.\n\nUsámos `ZeroExt` para representar cada operando com `2*n` bits e
definir\n`E: wide(x)*wide(y)+wide(z)==wide(a)*wide(b)`.\nEsta igualdade é
exata: mesmo o maior valor possível de `xy+z` cabe em `2*n` bits.\nO
invariante de segurança reúne `E`, `Q`, os limites das entradas e a
ausência de erro.\n\nAs consultas `Safety T5` e `Safety T7` mostram que,
sob esse invariante, as\ntransições para erro não podem ocorrer. Em T6 e
T8, provamos que as atualizações\nestendidas são exatas e usamos novamente
as identidades algébricas.\n`SafeInv ∧ Error` é também consultado; este
último teste confirma a exclusão\nexpressa no invariante, enquanto a base e
a preservação justificam que ele vale\nnos estados alcançáveis.\n\nQuando
chegamos a `pc=6`, temos `y=0`; logo `E` implica **z=a*b**, como
igualdade\nmatemática, para as entradas restringidas.\n\n'),
mo.md("```python\n" + blocos_codigo[3] + "\n```")])
```

In [ ]:
```
mo.vstack([mo.md('## D — Terminação\n\nA função de ranking usa inteiros sem
overflow:\n\n$$\\rho=\\begin{cases}0&pc\\in\\
{5,6,7\\},\\\\\n3\\,\\operatorname{BV2Int}(y)+fase(pc)&\\text{caso
contrário},\\end{cases}$$\n\ncom fases 3 na entrada, 2 no teste e 1 nas
atualizações.\n`BV2Int(y)` interpreta `y` sem sinal. As consultas mostram
que o domínio de controlo\ne `Q` são preservados, que o ranking é não
negativo e que diminui em cada transição\nque parte de um estado não
terminal. A fase permite a descida mesmo quando `y`\nainda não muda, como
na passagem do teste para uma atualização.\n\nAs guardas cobrem os estados
válidos: no teste, `y` é zero, par não nulo ou ímpar;\nnas atualizações,
ocorre overflow ou não. Assim, não há bloqueios nesses estados.\nUma
sequência estritamente decrescente nos naturais termina, pelo que se
atinge\n`pc=5`, `6` ou `7`. Com as hipóteses de segurança, os erros são
excluídos e a\nterminação é normal: **eventualmente pc=6**.'),
mo.md("```python\n" + blocos_codigo[4] + "\n```")])
```

In [ ]:
```
mo.vstack([mo.md('## Resultado global da verificação\n\nDepois das provas anteriores, `all(resultados)` confirma que todas as obrigações foram demonstradas. A função devolve `True` apenas nesse caso; caso contrário, indica que a verificação está incompleta.'), mo.md("```python\n" + blocos_codigo[5] + "\n```")])
```

Executar e consultar os testes

O botão executa a função `verificar` incluída neste documento, apresentada nos blocos anteriores. Não é necessário ter o ficheiro `exe3.py` ao lado para executar as provas. O limite de tempo aplica-se a **cada consulta**, não ao conjunto completo. Larguras maiores podem demorar mais ou produzir `unknown`.

- **unsat**: não existe contraexemplo à obrigação consultada; a obrigação foi provada.
- **sat**: existe contraexemplo; o programa mostra o modelo do Z3.
- **unknown**: a prova foi inconclusiva; o programa mostra a razão, por exemplo timeout.

A função só devolve `True` se todas as obrigações resultarem em `unsat`.

In [ ]:
```
larguras = mo.ui.multiselect([2, 4, 8, 16, 32], value=[8, 16], label="Larguras em bits")
timeout = mo.ui.number(start=100, stop=60000, step=100, value=10000, label="Timeout por consulta (ms)")
executar = mo.ui.run_button(label="Executar verificação")
mo.vstack([larguras, timeout, executar])
```

In [ ]:
```
mo.stop(not executar.value, mo.md("Carrega em **Executar verificação** para obter novos resultados."))
mo.stop(not larguras.value, mo.md("Seleciona pelo menos uma largura."))
linhas_resultado = []
saidas_resultado = []
for _n in larguras.value:
    _saida = io.StringIO()
    _inicio = time.perf_counter()
    with contextlib.redirect_stdout(_saida):
        _sucesso = verificar(_n, timeout_ms=int(timeout.value))
    _texto = _saida.getvalue()
    linhas_resultado.append({"n (bits)": _n, "unsat": sum(_l.endswith(": unsat") for _l in _texto.splitlines()), "sat": sum(_l.endswith(": sat") for _l in _texto.splitlines()), "unknown": sum(_l.endswith(": unknown") for _l in _texto.splitlines()), "Tempo (s)": round(time.perf_counter()-_inicio, 3), "Conclusão": "Todas provadas" if _sucesso else "Verificação incompleta"})
    saidas_resultado.append(mo.md("### n = " + str(_n) + "\n\n```text\n" + _texto + "\n```"))
mo.vstack([mo.ui.table(linhas_resultado, selection=None), *saidas_resultado])
```

Análise dos resultados de referência

A execução de `exe3.py`, em 3 de novembro de 2026, com timeout de 10000 ms por consulta, concluiu com sucesso para as duas larguras. A saída integral está em `resultados/exe3_testes.txt`.

Grupo	Obrigações por largura	n=8	n=16
Invariante: base, transições e composição	13	Todas unsat	Todas unsat
Segurança: base, transições, composição e exclusão de erro	14	Todas unsat	Todas unsat
Terminação: controlo e ranking	4	Todas unsat	Todas unsat
Total	**31**	**31 unsat**	**31 unsat**

Foram provadas **62 obrigações** no total. Não houve `sat` nem `unknown`. O sucesso significa que as fórmulas consultadas não admitem contraexemplos. Não corresponde apenas a experimentar alguns pares de entradas: as variáveis são simbólicas e abrangem os valores permitidos pelas hipóteses de cada consulta.

A segurança e o resultado exato aplicam-se às entradas limitadas indicadas na alínea C. O invariante modular e a terminação até um estado terminal têm hipóteses mais gerais. Cada chamada prova apenas a largura escolhida: estes resultados não constituem uma prova única para todos os valores possíveis de `n`. A composição de T6 e T8 e a cobertura das guardas devem ser consideradas juntamente com as respostas do solver.

In [ ]:
```
_ficheiro = pasta_exe3 / "resultados" / "exe3_testes.txt"
_registo = _ficheiro.read_text(encoding="utf-8") if _ficheiro.exists() else
"Registo não encontrado. Executa os testes no botão acima."
mo.accordion({"Saída integral dos testes de referência": mo.md("```text\n"
+ _registo + "\n```")})
```